

Distributed Collatz System Documentation

1. Architecture

The system consists of two ROS 2 nodes on separate machines communicating via a TCP socket server:

- **C++ Node (Client):**
 - Starts by selecting a random starting number (1–1,000,000).
 - Applies the Collatz “odd” rule: if the number is odd and not 1, compute $3n + 1$ in a loop until the result is even or reaches 1.
 - Sends the current number to the Python node via a TCP socket connection.
 - Receives the updated number back from the Python node and repeats until the sequence terminates at 1.
- **Python Node (Server):**
 - Binds to port 5000 and listens for an incoming connection from the C++ client.
 - Upon connection, receives a number.
 - Applies the Collatz “even” rule: while the number is even and not 1, divide by 2 in a loop.
 - Sends the result back to the C++ client.
 - Continues this receive-process-send cycle until the Collatz sequence completes (number becomes 1), then closes the connection.

Communication flows strictly via TCP sockets—no ROS 2 publishers, subscribers, or executors are used, and both nodes avoid `rclcpp::spin` or `rclpy.spin` loops, relying instead on manual loop control.

2. Challenges

- **Communication Method:**
 - Deciding between native ROS 2 messaging vs. direct sockets. ROS 2’s executor model and spinning loops simplify message passing, but the task forbade using any `spin()` method, making pure ROS 2 communication impractical.
 - Implementing reliable reconnection and error handling in socket code required careful management of blocking calls and socket lifetimes.
-

3. Key Design Choices

1. Socket Programming over ROS 2 Messaging:

- **Reason:** To honor the requirement of no `spin()`, sockets provide a straightforward blocking/non-blocking API for send/receive operations in user-controlled loops.
- **Benefit:** Full control over the communication loop; easier to visualize the data flow.

2. Manual Looping in Nodes:

- **Reason:** Avoid ROS 2 executors and `spin()` calls.
 - **Benefit:** Simplifies the node's flow into a clear sequence: compute → send → receive → repeat.
-

4. Lessons Learned

- **ROS 2 Executors:** Gained familiarity with ROS 2's multi-threaded and single-threaded executors, even though they were not used.
- **Socket Programming:** Reinforced understanding of POSIX sockets in C++ and Python.
- **Cross-Platform Networking:** Learned to configure VirtualBox network modes, host-to-VM bridging, and firewall rule management.
- **Git Workflow:** Improved repository versioning, branching, and remote management for collaborative ROS projects.
- **ROS_DOMAIN_ID:** Explored isolation of ROS 2 networks using unique domain IDs when running multiple ROS installations on one host.